ЛР 2. Программная реализация алгоритма выполнения целочисленной операции для архитектуры набора команд ARM.

Вариант алгоритма выбирается в соответствии с ЛР1.

В процессе самостоятельной работы:

1. Выбрать исходные данные в соответствии с номером варианта.
2. Изучить набор команд ARM по справочным материалам, которые приложены к заданию.
3. Изучить примеры программ из папки ArmSim и запустить их в одноимённом симуляторе.
4. Выбрать места для хранения данных: регистры или память.
5. Взять первый блок в блок-схеме из ЛР1 и написать для него команды на ассемблере ARM. Если для какого-то блока трудно найти подходящие команды – задать вопрос преподавателю.
6. Повторить шаг 5 для всех блоков из блок-диаграммы.
7. Отладить код в симуляторе архитектуры ARM.
8. Предъявить отчет преподавателю в электронном виде. Защитить отчет.

Содержание отчета:

1. Титульный лист.
2. Текст задания.
3. Блок-схема алгоритма.
4. Код программы на языке ассемблера ARM.
5. Карта распределения памяти под программу и данные (где и по каким адресам храним).
6. Скриншот окна симулятора с примером вычислений.

Организация ЭВМ

Лабораторная работа №2

Программная реализация алгоритма выполнения целочисленной операции для архитектуры набора команд ARM

Целью работы является изучение набора команд архитектуры ARM. Изучить набор команд можно по прилагающимся материалам, а так же из источников в интернете. Результатом выполнения работы станет программа на ассемблере.

В качестве примера рассмотрим алгоритм, некий алгоритм умножения:

Первым делом составим список переменных:
A - множимое (знак + 8 разрядов);
B - множитель (знак + 4 разряда);
S – сумма частичных произведений (знак + 8 разрядов);
c - бит переполнения (для анализа очередного бита B, возможны другие варианты, например маскирование, зависит от процессора и системы команд);
b - бит для хранения знака числа B;
i - счётчик цикла.

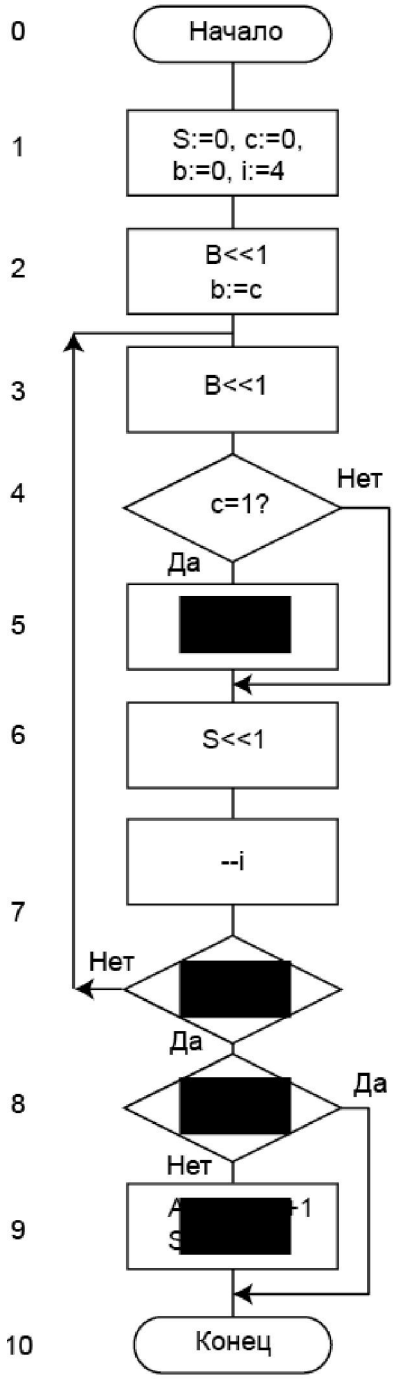
Распределим переменные по регистрам общего назначения (РОН) процессора:

РОН	Переменная
r0	A - множимое
r1	B - множитель
r2	S – сумма частичных произведений
r3	i - счётчик цикла
r4	Маска (об этом позже)
	для b, я не стал выделять место, так как внёс изменения в алгоритм, о чём будет сказано ниже

Справка: В любом режиме работы процессора имеется доступ к следующим регистрам:
– r0-r12 РОН
– r13 (the stack pointer, sp) иr14 (the link register, lr)
– program counter, r15 (pc)
– current program status register, cpsr
Смело используйте регистры r0 – r10 на своё усмотрение, как не имеющие дополнительных функций.

Разрядность регистров: 32 бита.

Теперь вспомним сам алгоритм:



Пошагово, реализуем каждый его блок.

Для примера будем умножать числа: A =13и B= -10, переведём их в требуемый формат:

$A_{доп}$ =0000 0000 0000 0000 0000 0000 0000 1101_2 = 0x0000000D_{16}

$B_{доп}$ =1111 1111 1111 1111 1111 1111 1111 0110_2 = 0xFFFFFFF6_{16}

Примечание: жёлтым отмечен знаковый разряд.

Блок 1, представляет собой набор присвоений (блок инициализации), реализуем его при помощи команды mov:

```
mov r0,#0x0000000d @загрузить в r0 шестнадцатеричную константу A, для примера
mov r1,#0xfffffff6 @множитель
mov r2,#0      @сумма частичных произведений
mov r3,#4      @счётчик цикла
mov r4,#8      @маска, для анализа старшего бита множителя (1000)
```

Блок 2, отвечает за сохранение знака множителя в отдельную переменную, это мера объяснялась тем, что алгоритм предполагал анализ битов множителя, путём выдавливания их в бит переполнения или бит знака, тем самым множитель разрушался и проанализировать его знак было нельзя. В результате изучения системы команд, было установлено, что выбранный метод реализовать сложнее, чем не менее популярный метод маскирования (тем не менее, вы можете использовать любой известный вам подход).

Справка: Метод маскирования

Суть метода маскирования заключается в скрытии «не нужной» информации при помощи «маски». Допустим, мы хотим узнать чему равен пятый бит в двоичном слове A = xxxxxxxx, не имея команд работы с битами, произведём логическую операцию «И» слова A и слова-маски B = 00010000, результатом станет слово C = 000x0000, всё что осталось, проверить C на равенство нулю, если не равно, значит искомый бит 1, иначе 0. Данный метод широко известен и применяется для различных целей (а также, не только для работы с битами и двоичными словами).

И так, применение метода маскирования исключает необходимость блока 2, пропускаем его. Блоки 3 и 4, отвечают за анализ очередного бита множителя (B), начинаем со старшего бита, минуя знак, вместо выдавливания бита в перенос - накладываем маску:

```
tst r1, r4 @наложение маски (r1 AND r4), результат не сохраняется, остаются только флаги
```

На блок-схеме алгоритма видно, что к этому блоку происходит возврат, по этому отметим его меткой для перехода:

```
label1: tst r1, r4
```

После выполнения этой команды, сформируются признаки (флаги), нас интересует флаг Z- равенство нулю. Если анализируемый бит единица, выполнятся блок 5, иначе 6. Условная команда перехода формируется из безусловного перехода b и признака, в данном случае eq – Z=0:

```
beq label2     @переход, если флаг Z=1

...
```

Следующий блок 6, выполняющий сдвиг частичных произведений влево, так же необходимо сдвинуть маску в право, для анализа следующего бита B:

```
label2: mov r2,r2,lsl #1@сдвиг суммы частичных произведений влево, на разряд
```

mov r4,r4,lsr #1@сдвиг маски вправо - для анализа следующего бита множителя

Эта часть программы отмечена меткой label2, так как сюда осуществляется переход в результате команды beq.

Справка: Сдвиги в ARM

В наборе команд ARM, нет команд осуществляющих сдвиги на прямую, вместо этого, второй операнд в командах подаётся в АЛУ, через сдвиговый регистр, в результате чего может быть сдвинут на указанное число бит mov r2,r2,lsl #1 – эта запись, означает, что в регистр r2 будет помещено значение регистра r2 со сдвигом на один бит влево.

На данном этапе, наша программа охватывает, блоки с 0 по 6 и выглядит так:

```
        mov r0,#0x0000000d @множимое
        mov r1,#0xfffffff6 @множитель
        mov r2,#0       @сумма частичных произведений
        mov r3,#4       @счётчик цикла
        mov r4,#8       @маска, для анализа старшего бита множителя (1000)
label1: tst ,r1, r4     @наложение маски (r1 AND r4), результат не сохраняется, остаются только флаги
        beq label2      @переход, если флаг Z=0
        add r2,r2,r0    @S+A
label2: mov r2,r2,lsl #1@сдвиг суммы частичных произведений влево, на разряд
        mov r4,r4,lsr #1@сдвиг маски вправо - для анализа следующего бита множителя
```

...
...
...

Заканчиваем программу строчкой:

```
finish:  .end           @конец, результат в r2
```

Загружаем программу и пробуем отладить, исправляем возникающие ошибки. К примеру, в данной программе и алгоритме соответственно, будет обнаружен лишний сдвиг суммы частичных произведений, исправляем ошибку:

```
        mov r0,#0x0000000d @множимое
        mov r1,#0xfffffff6 @множитель
        mov r2,#0       @сумма частичных произведений
        mov r3,#4       @счётчик цикла
        mov r4,#8       @маска, для анализа старшего бита множителя (1000)
label1: mov r2,r2,lsl #1@эта строчка перенесена, раньше она была после label2, таким образом число сдвигов в цикле уменьшилось на 1, так как первый сдвиг пустой – в r2 пока ещё ноль
```

```
        tst r1, r4      @наложение маски (r1 AND r4), результат не сохраняется, остаются только флаги
        beq label2      @переход, если флаг Z=0
        ...
label2: mov r4,r4,lsr #1@сдвиг маски вправо - для анализа следующего бита множителя
        ...
        ...
        ...
finish: .end            @конец, результат в r2
```

Для отчёта потребуется заполнить таблицу:

Команда	Состояние памяти и регистров (указываем те, что изменяет команда)	
	Было	Стало
mov r0,#0x0000000d	r0 =0, pc(r15)=1000	r0 = 0000000d, pc=1004
mov r1,#0xfffffff6	r1 =0, pc=1004	r1= fffffff6, pc=1008
tst r1, r4	pc=1018, Z=0	pc=101c, Z=1
beq label2	pc=101c	pc=1024
И так далее, до конца программы (с заходом в циклы)		